

Pandas is a powerful Python library for data manipulation and analysis. It provides data structures like Series (1-dimensional) and DataFrame (2-dimensional) that are highly optimized for handling structured data.

Here's a breakdown of common data manipulation and transformation techniques in pandas with examples:

1. Creating DataFrames

You can create DataFrames from various sources, including dictionaries, lists, NumPy arrays, and CSV/Excel files.

```
import pandas as pd
import numpy as np

# From a dictionary
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40],
        'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print("-" * 30)
```

```
# From a list of lists
data_list = [['Alice', 25, 'New York'],
             ['Bob', 30, 'Los Angeles'],
             ['Charlie', 35, 'Chicago']]
df_list = pd.DataFrame(data_list, columns=['Name', 'Age', 'City'])
print("\nDataFrame from list of lists:")
print(df_list)
print("-" * 30)
```

2. Selecting Data

You can select rows, columns, or specific cells using various methods.

- Column Selection:

Python

```
print("Selecting 'Name' column:")
• print(df['Name'])
• print("-" * 30)
•
• print("\nSelecting multiple columns ('Name',
'City'):")
• print(df[['Name', 'City']])
• print("-" * 30)
•
```

- Row Selection (using loc and iloc):
 - loc: Label-based indexing (selects by row and column labels).
 - iloc: Integer-based indexing (selects by row and column integer positions).
- Python

```
print("\nSelecting first row using iloc:")
• print(df.iloc[0])
• print("-" * 30)
•
• print("\nSelecting rows with Age > 30 using boolean indexing:")
• print(df[df['Age'] > 30])
• print("-" * 30)
•
• print("\nSelecting rows with specific city using loc:")
• print(df.loc[df['City'] == 'New York'])
• print("-" * 30)
•
• print("\nSelecting specific cell (row 1, 'Age' column) using loc:")
• print(df.loc[1, 'Age'])
• print("-" * 30)
•
• print("\nSelecting specific cell (row 1, column 1) using iloc:")
• print(df.iloc[1, 1])
• print("-" * 30)
•
```

3. Adding/Deleting Columns Python

```
# Adding a new column
df['Salary'] = [50000, 60000, 75000, 80000]
print("\nDataFrame after adding 'Salary' column:")
print(df)
print("-" * 30)
```

```
# Deleting a column
df_dropped = df.drop(columns=['City'])
print("\nDataFrame after dropping 'City' column:")
```

```

print(df_dropped)
print("-" * 30)

# Deleting a column in-place (modifies original DataFrame)
df.drop(columns=['Salary'], inplace=True)
print("\nDataFrame after dropping 'Salary' column (in-place):")
print(df)
print("-" * 30)

```

4. Handling Missing Data

Pandas uses NaN (Not a Number) to represent missing values.

Python

```

data_missing = {'A': [1, 2, np.nan, 4],
                 'B': [5, np.nan, 7, 8],
                 'C': [9, 10, 11, 12]}
df_missing = pd.DataFrame(data_missing)
print("\nDataFrame with missing values:")
print(df_missing)
print("-" * 30)

# Check for missing values
print("\nMissing values check:")
print(df_missing.isnull())
print("-" * 30)

# Drop rows with any missing values
df_cleaned_rows = df_missing.dropna()
print("\nDataFrame after dropping rows with any missing values:")
print(df_cleaned_rows)
print("-" * 30)

# Fill missing values with a specific value
df_filled = df_missing.fillna(0)
print("\nDataFrame after filling missing values with 0:")
print(df_filled)
print("-" * 30)

# Fill missing values with the mean of the column
df_filled_mean = df_missing.fillna(df_missing['B'].mean())
print("\nDataFrame after filling missing 'B' values with its mean:")
print(df_filled_mean)
print("-" * 30)

```

5. Applying Functions (apply, map, applymap)

- apply: Apply a function along an axis of the DataFrame or Series.
- map: Element-wise mapping on a Series.
- applymap: Element-wise mapping on a DataFrame.

Python

```

# Using apply on a column (Series)

```

```

df['Age_squared'] = df['Age'].apply(lambda x: x**2)
print("\nDataFrame with 'Age_squared' column (using apply):")
print(df)
print("-" * 30)

# Using map on a Series (e.g., to categorize)
gender_mapping = {'Alice': 'Female', 'Bob': 'Male', 'Charlie':
'Male', 'David': 'Male'}
df['Gender'] = df['Name'].map(gender_mapping)
print("\nDataFrame with 'Gender' column (using map):")
print(df)
print("-" * 30)

# Using applymap on the entire DataFrame (for numeric data)
df_numeric = df[['Age', 'Age_squared']]
df_numeric_plus_one = df_numeric.applymap(lambda x: x + 1)
print("\nNumeric DataFrame with each element incremented by 1 (using
applymap):")
print(df_numeric_plus_one)
print("-" * 30)
6. Grouping and Aggregating (groupby)
groupby is crucial for performing operations on subsets of your
data.
Python

```

```

data_sales = {'Region': ['East', 'West', 'East', 'West', 'East'],
               'Product': ['A', 'B', 'A', 'C', 'B'],
               'Sales': [100, 150, 120, 80, 200]}
df_sales = pd.DataFrame(data_sales)
print("\nSales DataFrame:")
print(df_sales)
print("-" * 30)

```

```

# Group by 'Region' and calculate sum of 'Sales'
sales_by_region = df_sales.groupby('Region')['Sales'].sum()
print("\nTotal Sales by Region:")
print(sales_by_region)
print("-" * 30)

```

```

# Group by multiple columns and get multiple aggregations
multi_agg = df_sales.groupby(['Region', 'Product']).agg(
    Total_Sales=('Sales', 'sum'),
    Average_Sales=('Sales', 'mean'),
    Count=('Sales', 'count')
)
print("\nMultiple Aggregations by Region and Product:")
print(multi_agg)
print("-" * 30)

```

7. Merging, Joining, and Concatenating DataFrames

- concat: Stacking DataFrames vertically or horizontally.
- merge: Combining DataFrames based on common columns

(like SQL joins).

- join: Similar to merge, but defaults to joining on index.
- Python

```
df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'], 'value': [1, 2, 3, 4]})
df2 = pd.DataFrame({'key': ['C', 'D', 'E', 'F'], 'value': [5, 6, 7, 8]})
df3 = pd.DataFrame({'key': ['A', 'B', 'G', 'H'], 'value': [9, 10, 11, 12]})
```

```
print("\nDataFrame 1:")
print(df1)
print("\nDataFrame 2:")
print(df2)
print("\nDataFrame 3:")
print(df3)
print("-" * 30)
```

```
# Concatenate vertically
df_concat_vert = pd.concat([df1, df2])
print("\nConcatenated DataFrames (vertically):")
print(df_concat_vert)
print("-" * 30)
```

```
# Concatenate horizontally
df_concat_horiz = pd.concat([df1, df3], axis=1) # Note: Be careful
with horizontal concat if indices don't align
print("\nConcatenated DataFrames (horizontally - caution if indices
don't align):")
print(df_concat_horiz)
print("-" * 30)
```

```
# Merge (inner join - default)
df_merged_inner = pd.merge(df1, df2, on='key', suffixes=('_df1',
'_df2'))
print("\nMerged DataFrames (inner join on 'key'):")
print(df_merged_inner)
print("-" * 30)
```

```
# Merge (left join)
df_merged_left = pd.merge(df1, df2, on='key', how='left',
suffixes=('_df1', '_df2'))
print("\nMerged DataFrames (left join on 'key'):")
print(df_merged_left)
print("-" * 30)
```

8. Pivoting and Melting

- pivot_table: Reshapes data based on column values, creating a "pivot" table.
- melt: Transforms wide format data into long format.

Python

```

data_pivot = {'Date': ['2023-01-01', '2023-01-01', '2023-01-02',
'2023-01-02'],
              'City': ['New York', 'Los Angeles', 'New York', 'Los
Angeles'],
              'Temperature': [10, 15, 12, 18],
              'Humidity': [70, 60, 65, 55]}
df_pivot = pd.DataFrame(data_pivot)
print("\nOriginal DataFrame for pivoting:")
print(df_pivot)
print("-" * 30)

```

```

# Pivot table to show Temperature by Date and City
pivot_table_temp = df_pivot.pivot_table(index='Date',
columns='City', values='Temperature')
print("\nPivot Table (Temperature by Date and City):")
print(pivot_table_temp)
print("-" * 30)

```

```

# Melt the DataFrame to long format
df_melted = df_pivot.melt(id_vars=['Date', 'City'],
var_name='Metric', value_name='Value')
print("\nMelted DataFrame:")
print(df_melted)
print("-" * 30)

```

9. Handling Duplicates Python

```

data_duplicates = {'A': [1, 2, 2, 3, 4, 4],
                  'B': ['X', 'Y', 'Y', 'Z', 'W', 'W']}
df_duplicates = pd.DataFrame(data_duplicates)
print("\nDataFrame with duplicates:")
print(df_duplicates)
print("-" * 30)

```

```

# Check for duplicates
print("\nDuplicate rows check:")
print(df_duplicates.duplicated())
print("-" * 30)

```

```

# Drop duplicate rows (keeps first occurrence by default)
df_no_duplicates = df_duplicates.drop_duplicates()
print("\nDataFrame after dropping duplicates:")
print(df_no_duplicates)
print("-" * 30)

```

```

# Drop duplicates based on a specific column
df_no_duplicates_A = df_duplicates.drop_duplicates(subset=['A'])
print("\nDataFrame after dropping duplicates based on column 'A':")
print(df_no_duplicates_A)

```

```
print("-" * 30)
10. Sorting Data
Python
```

```
print("\nOriginal DataFrame:")
print(df) # Using the initial 'df' which has Name, Age, City,
Gender, Age_squared
print("-" * 30)
```

```
# Sort by 'Age' in ascending order
df_sorted_age = df.sort_values(by='Age')
print("\nDataFrame sorted by 'Age' (ascending):")
print(df_sorted_age)
print("-" * 30)
```

```
# Sort by 'Age' in descending order and then by 'Name' in ascending
order
df_sorted_multi = df.sort_values(by=['Age', 'Name'],
ascending=[False, True])
print("\nDataFrame sorted by 'Age' (descending) and then 'Name'
(ascending):")
print(df_sorted_multi)
print("-" * 30)
```

This comprehensive overview covers many of the fundamental data manipulation and transformation techniques you'll use with pandas. Mastering these operations is key to effectively cleaning, preparing, and analyzing your data.